COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Baltazar, Kenji Formilleza, Huone Anthony	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 **Algorithmic Complexity**

1. PROCEDURE OUTPUT

	Best Case [Time]	Worse Case [Time]	Best Case [Space]	Worse Case [Space]	Analysis
Insertion Sort	O(n)	$O(n^2)$	O(1)	O(1)	If this algorithm were to sort having an input of 100, it would be fastly executed compared to shell short due to it being O(n) only, but the worst time complexity could potentially be much more than shell sort due to it being a lower version of sorting algorithm than shell sort.
Shell Sort	O(n log n)	$O(n^2)$	O(1)	O(1)	In this algorithm shell sort is a stable one, Its best case is O of n log n and its worst case is O of n squared, with O of 1 space. But when we actually ran it, the times at n of 10000 were just 386 microseconds for the best case and 540 for the worst case, so not really a big gap. It seems the worst case never really got close to n squared, probably because of the gap sequence we used.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

CODE

```cpp
#include <iostream>
#include <chrono>
#include "sortingAlgo.h"
#include "sortingAlgo2.h"

void sort_test(int size) {
    int* a = new int[size];
    int* b = new int[size];

    for (int i = 0; i < size; i++) {
        a[i] = i;
        b[i] = i;
    }

    auto begin = std::chrono::steady_clock::now();
    insertionSort(a, size);
    auto end = std::chrono::steady_clock::now();
    std::cout << "n = " << size << ", Best: Insertion: "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds";

    begin = std::chrono::steady_clock::now();
    shellSort(b, size);
    end = std::chrono::steady_clock::now();
    std::cout << ", Shell: "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    for (int i = 0; i < size; i++) {
        a[i] = size - i;
```

```
        b[i] = size - i;
    }

    begin = std::chrono::steady_clock::now();
    insertionSort(a, size);
    end = std::chrono::steady_clock::now();
    std::cout << "n = " << size << ", Worst: Insertion: "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds";

    begin = std::chrono::steady_clock::now();
    shellSort(b, size);
    end = std::chrono::steady_clock::now();
    std::cout << ", Shell: "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl << std::endl;
}

int main() {
    sort_test(100);
    sort_test(1000);
    sort_test(10000);
    return 0;
}
```

OUTPUT

Baltazar

```
n = 100, Best: Insertion: 0 microseconds, Shell: 2 microseconds
n = 100, Worst: Insertion: 12 microseconds, Shell: 3 microseconds

n = 1000, Best: Insertion: 3 microseconds, Shell: 26 microseconds
n = 1000, Worst: Insertion: 1131 microseconds, Shell: 39 microseconds

n = 10000, Best: Insertion: 31 microseconds, Shell: 388 microseconds
n = 10000, Worst: Insertion: 116065 microseconds, Shell: 540 microseconds
```

Formilleza

```
n = 100, Best: Insertion: 0 microseconds, Shell: 1 microseconds
n = 100, Worst: Insertion: 12 microseconds, Shell: 3 microseconds

n = 1000, Best: Insertion: 3 microseconds, Shell: 25 microseconds
n = 1000, Worst: Insertion: 1110 microseconds, Shell: 39 microseconds

n = 10000, Best: Insertion: 31 microseconds, Shell: 386 microseconds
n = 10000, Worst: Insertion: 114864 microseconds, Shell: 540 microseconds
```

Analysis		
	Insertion Sort	Shell Sort
Best Case	According to the executed output, the best case output 0, 3 and 31 microseconds beating shell sort in terms of having a best case scenario.	As seen on the output above, the best case would be 1-2, 25-26, and 386-388 microseconds which is lagging behind in insertion sort by quite a large margin, especially in a situation having 10000 input.
Worst Case	It took 114864-116065 microseconds at n of 10000, which is really slow compared to everything else in our results. When the data is in reverse order, every element has to be	3, 39, 540 It only took 540 microseconds at n of 10000, which is way faster than Insertion Sort. It moves elements across big gaps first, so the data gets

	shifted all the way to the front, one step at a time. The time also jumped about 100 times every time n grew 10 times, so it gets worse very fast as the data gets bigger.	close to sorted early on and the last passes have very little left to fix. Its time only grew a little each time n got bigger, so it handles large data well.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. SUPPLEMENTARY

Pseudocode Analysis

Linear	Binary
LinearSearch(list, target): for i from 0 to n-1: if list[i] == target: return found return not found	BinarySearch(list, target): first = 0, last = n while range is not empty: mid = middle of [first, last) if list[mid] == target: return found else if list[mid] > target: last = mid else: first = mid + 1 return not found

Time and Space Complexities

	Linear Search	Binary Search
Best case	O(1)	O(1)
Worst case	O(n)	O(log n)
Average case	O(n), about n/2 checks	O(log n)
Space	O(1)	O(1) iterative, O(log n) if recursive
Needs sorted data?	No	Yes

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	160 ns	385 ns
10,000	188 ns	4,333 ns

100,000	233 ns	59,909 ns

3. CONCLUSION

BALTAZAR	FORMILLEZA
In this activity, I further delved deeper to understand different kinds of algorithms for searching and sorting. I was also able to compare the best and worse time complexity of two kinds of sorting and searching algorithms. In addition, I also learned from this that although there are more advanced types of sorting algorithms, they are still some downsides compared to some simpler ones. Like in shell sort, it is more unstable than insertion sort because it uses a gap technique, but due to this fact as well, this makes it much more suited for larger data types when the worse case scenario involves. Proof of this is the output in the procedure where there is an overwhelming large gap between those two worse cases. In addition, I was also able to compare binary and linear searches in the supplementary. Overall, I learned many things and was able to recall past lessons during this activity.	In this activity, I learned that the order of the data really matters, insertion sort was much faster in the best case scenario at 31 microseconds for n of 10000, but it got really slow in the worst case at 114864-116065 microseconds. Shell Sort was slower on sorted data at 386 microseconds, but it stayed fast in the worst case at only 540 microseconds. The results also matched the theory. Insertion Sort grew like O of n in the best case and O of n squared in the worst case, while Shell Sort grew slowly in both. So Insertion Sort is good for small or almost sorted data, and Shell Sort is better for large data or when the order is unknown.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion

of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.